AI ASSISTED CODING

ASSIGNMENT - 13.5

NAME: R.Teeneswari

HL-NO: 2303A51932 **BATCH:** 30

Lab 13.5:

Task-1:

```
def calculate_rectangle_metrics(length, width):
    """
    Calculate and print the area and perimeter of a rectangle.
    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    None
    """
    area = length * width
    perimeter = 2 * (length + width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
    print("-" * 30)
def main():
    """
    Main function to call rectangle calculations
    for multiple rectangle dimensions.
    """
    rectangles = [
        (5, 10),
        (7, 12),
        (10, 15)
    ]
    for length, width in rectangles:
        calculate_rectangle_metrics(length, width)
if __name__ == "__main__":
```

```
    main()
```

Output:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
------------------------------
Area of Rectangle: 84
Perimeter of Rectangle: 38
------------------------------
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task-2:

```
def calculate_total(price: float, tax_rate: float = 0.18) -> float:
    """
    Calculates the total price including tax.

    Args:
        price (float): The base price of the item.
        tax_rate (float): The tax rate to apply. Defaults to 0.18.
    Returns:
        float: The total price after adding tax.
    """
    tax = price * tax_rate
    return price + tax
def main():
    # List of prices to process
    prices = [250, 500]
    for price in prices:
        total = calculate_total(price)
        print(f"Total Price: {total}")
if __name__ == "__main__":
    main()
```

Output:

```
Total Price: 295.0
Total Price: 590.0
```

Task-3:

```python
class GradeCalculator:
    """
    A class used to calculate student grades based on marks.
    """
    def calculate_grade(self, marks):
        """
        Calculates the grade for a given mark.

        Parameters:
        marks (int or float): The student's marks (0-100).
        Returns:
        str: The corresponding grade description.
        """
        if 90 <= marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        else:
            return "Fail"
# Create an object of GradeCalculator
calculator = GradeCalculator()
# Test cases
test_marks = [85, 72, 95, 45, 30]
for marks in test_marks:
    grade = calculator.calculate_grade(marks)
    print(f"Marks: {marks} -> {grade}")
```

Output:

```
Marks: 85 -> Grade B
Marks: 72 -> Grade C
Marks: 95 -> Grade A
Marks: 45 -> Grade D
Marks: 30 -> Fail
```

Task-4:

```python
def get_input():
    return int(input("Enter number: "))


def calculate_square(num):
    return num * num
def display_result(square):
    print("Square:", square)
def main():
    num = get_input()
    square = calculate_square(num)
    display_result(square)
if __name__ == "__main__":
    main()
```

Output:

```
Enter number: 2
Square: 4
```

Task-5:

```python
class EmployeeSalaryCalculator:
    def __init__(self, salary, tax_rate=0.2):
        self.salary = salary
```

```python
        self.tax_rate = tax_rate


    def calculate_tax(self):
        return self.salary * self.tax_rate
    def calculate_net_salary(self):
        return self.salary - self.calculate_tax()
    def display_net_salary(self):
        print(self.calculate_net_salary())
if __name__ == "__main__":
    calculator = EmployeeSalaryCalculator(50000)
    calculator.display_net_salary()
```

Output:

```
40000.0
```

Task-6:

```python
# Task 6 (Optimizing Search Logic)
# Refactored search logic using a set for O(1) average time complexity


# Legacy Code used a list O(n) search time.
# A set is chosen because lookups are O(1) on average.
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
# Set lookup is efficient and scales well with large datasets
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Output:

```
Enter username: sai
Access Denied
```

Task-7:

```python
"""
Library Management System Module (OOP Version)

This module provides a Library class to manage books
using object-oriented programming principles.
"""
class Library:
    """
    A class to represent a library management system.
    """
    def __init__(self):
        """
        Initialize the library with an empty database.
        """
        self.library_db = {}
    def add_book(self, title, author, isbn):
        """
        Add a new book to the library.
        Parameters:
            title (str): Title of the book.
            author (str): Author of the book.
            isbn (str): Unique ISBN number.
        Returns:
            str: Success or error message.
        """
        if isbn not in self.library_db:
            self.library_db[isbn] = {
                "title": title,
                "author": author
            }
            return "Book added successfully."
```

```
        return "Book already exists."
    def remove_book(self, isbn):
        """
        Remove a book from the library.
        Parameters:
            isbn (str): ISBN of the book to remove.
        Returns:
            str: Success or error message.
        """
        if isbn in self.library_db:
            del self.library_db[isbn]
            return "Book removed successfully."
        return "Book not found."
    def search_book(self, isbn):
        """
        Search for a book using ISBN.
        Parameters:
            isbn (str): ISBN of the book to search.
        Returns:
            dict or str: Book details if found, otherwise error message.
        """
        if isbn in self.library_db:
            return self.library_db[isbn]
        return "Book not found."
```

Output:

```
NAME
    library - This module provides a simple Library Management System for managing a collection of books.

FUNCTIONS
    add_book(title, author, isbn)
        Adds a new book to the library database.

        Args:
            title (str): The title of the book.
            author (str): The author of the book.
            isbn (str): The ISBN of the book.

        Returns:
            str: Success or failure message.

    remove_book(isbn)
        Removes a book from the library database by its ISBN.

        Args:
            isbn (str): The ISBN of the book to remove.

        Returns:
            str: Success or failure message.

    search_book(isbn)
        Searches for a book in the library database by its ISBN.

        Args:
            isbn (str): The ISBN of the book to search for.

        Returns:
            dict or str: The book details if found, otherwise a 'Book not found' message.

DATA
    library_db = {}

FILE
```

index

library c:\users\sai\onedrive\attachments\ai assisted coding\library.py

This module provides a simple Library Management System for managing a collection of books.

Functions

add_book(title, author, isbn)
Adds a new book to the library database.

Args:
title (str): The title of the book.
author (str): The author of the book.
isbn (str): The ISBN of the book.

Returns:
str: Success or failure message.

remove_book(isbn)
Removes a book from the library database by its ISBN.

Args:
isbn (str): The ISBN of the book to remove.

Returns:
str: Success or failure message.

search_book(isbn)
Searches for a book in the library database by its ISBN.

Args:
isbn (str): The ISBN of the book to search for.

Returns:
dict or str: The book details if found, otherwise a 'Book not found' message.

Data

library_db = {}

Task-8:

```
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n terms.

    Parameters:
        n (int): Number of Fibonacci terms to generate.
    Returns:
```

```python
        list: A list containing Fibonacci sequence up to n terms.
    Raises:
        ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("Number of terms cannot be negative.")
    if n == 0:
        return []
    if n == 1:
        return [0]
    fibonacci_series = [0, 1]
    for _ in range(2, n):
        fibonacci_series.append(
            fibonacci_series[-1] + fibonacci_series[-2]
        )
    return fibonacci_series
# ---------------- TEST CASES ---------------- #
if __name__ == "__main__":
    print("Test Case 1 (n=5):", generate_fibonacci(5))
    print("Test Case 2 (n=1):", generate_fibonacci(1))
    print("Test Case 3 (n=0):", generate_fibonacci(0))
```

Output:

```
PS C:\Users\SAI\OneDrive\Attachments\AI ASSISTED CODING>
Test Case 1 (n=5): [0, 1, 1, 2, 3]
Test Case 2 (n=1): [0]
Test Case 3 (n=0): []
```

```
Password too short
```